KRR Project — Milestone 3

AudiovisualOntology

Full Project Documentation

ANKRI Mohamed-Khalil · MICHON Charlotte · PAULIS Antoine

Academic Year 2025–2026

Table of Contents

1. Project Overview

This document describes the complete implementation of Milestone 3 of the Knowledge Representation and Reasoning (KRR) project. The goal of this milestone was to transform a relational SQL database — sourced from IMDb data — into a fully functional RDF knowledge graph using the R2RML mapping language, validate the graph using SHACL constraints, and demonstrate its utility through a collection of SPARQL queries and an interactive web demonstrator.

The project pipeline consists of four main stages:

- **SQL → RDF Mapping:** An R2RML mapping file transforms the relational IMDb schema into RDF triples conforming to the AudiovisualOntology.
- **Ontology Alignment:** All generated individuals are typed and linked according to the classes and properties defined in the OWL ontology.
- **SHACL Validation:** A SHACL shapes file verifies that every individual satisfies the mandatory constraints defined for its class.
- **SPARQL Querying:** A set of 18 queries (8 simple, 5 medium, 5 non-trivial) demonstrates the full breadth of the knowledge graph.

1.1 Pipeline Diagram

Step	Input	Tool	Output
1	IMDb SQL database	phpMyAdmin / MariaDB	imdb.sql
2	imdb.sql + mapping.ttl	R2RML Mapper (r2rml.jar)	out.ttl
3	out.ttl + shapes.ttl	Apache Jena SHACL	Validation report
4	out.ttl	Apache Jena ARQ / Flask + rdflib	SPARQL results

2. Database Schema

The source database is a MariaDB relational schema containing IMDb data. It consists of 13 tables organised into three logical groups: reference/dictionary tables, core entity tables, and junction tables.

2.1 Reference Tables (Dictionaries)

These tables store controlled vocabulary values used as lookup values throughout the schema.

Table	Primary Key	Key Columns
category	category_id	category_name
content_type	content_type_id	content_type_name
genre	genre_id	genre_name
role	role_id	role_name
title_type	title_type_id	title_type_name
region	region_id	region_name
language	language_id	language_name

2.2 Core Entity Tables

Table	Primary Key	Key Columns
title	title_id	primary_title, original_title, start_year, end_year, runtime_minutes, content_type_id
talent	talent_id	talent_name, birth_year, death_year

2.3 Junction Tables

Table	Purpose
title_principal	Links a talent to a title with a category, job description, and role names (ord)
talent_role	Links a talent to their industry roles (actor, director, etc.)
talent_title	Direct talent-to-title association (separate from principal)
title_genre	Many-to-many: titles to genres
title_aka	Alternative/localised titles with region and language
title_aka_title_type	Links alternative titles to their title types
title_episode	TV episode metadata: season, episode number, parent title

3. Ontology Design

The AudiovisualOntology is defined in OWL and stored in audiovisual-ontology-M3.owx. It models the audiovisual domain using 10 classes and a set of object and data properties.

3.1 Classes

Class	Description	Maps to SQL table
Title	A film, TV show, episode, or any audiovisual work	title
Talent	A person involved in the creation of a title	talent
Participation	Reification bridge: a specific involvement of a talent in a title	title_principal
AlternativeTitle	A localised or alternative name for a title	title_aka
Category	Broad industry category (actor, director, writer...)	category
Role	Specific credited role (e.g. camera department, composer)	role
Genre	Genre of a title (Drama, Comedy, etc.)	genre
ContentType	Type of content (movie, TV Episode, short...)	content_type
Region	Geographic region associated with an alternative title	region
TitleType	Classification type of an alternative title	title_type

3.2 Key Object Properties

Property	Domain	Range	Description
isClassifiedAs	Title	ContentType	Links a title to its content type
contains	Title	Participation	Links a title to its participations
participatesIn	Talent	Participation	Links a talent to its participations
as	Participation	Category	Category of a participation
plays	Participation	Role	Role played in a participation
hasGenre	Title	Genre	Genre of a title
isAka	Title	AlternativeTitle	Alternative title of a title
isFrom	AlternativeTitle	Region	Region of an alternative title
hasType	AlternativeTitle	TitleType	Type of an alternative title

3.3 Key Data Properties

Property	Domain	Type
primaryTitle	Title	xsd:string
startYear	Title	xsd:string
talentName	Talent	xsd:string
roleName	Role	xsd:string
categoryName	Category	xsd:string
genreName	Genre	xsd:string
content_type_name	ContentType	xsd:string
regionName	Region	xsd:string
title_type_name	TitleType	xsd:string
alternativeTitleName	AlternativeTitle	xsd:string
job	Participation	xsd:string

4. R2RML Mapping

The R2RML mapping file (mapping_fixed_v2.ttl) defines how each SQL table is transformed into RDF triples. Each mapping is called a TriplesMap and consists of a logical table (the SQL source), a subject map (defining the IRI template and class), and one or more predicate-object maps (defining properties).

4.1 IRI Templates

All individuals use the base URI **http://www.semanticweb.org/antoi/data/resource/** followed by a class-specific path and the primary key value. For example:

Class	IRI Template
Title	.../title/{title_id}
Talent	.../talent/{talent_id}
Participation	.../participation/{title_id}_{talent_id}_{ord}
AlternativeTitle	.../alternativeTitle/{title_id}_{ord}
Category	.../category/{category_id}
Role	.../role/{role_id}
Genre	.../genre/{genre_id}
ContentType	.../contentType/{content_type_id}
Region	.../region/{region_id}
TitleType	.../titleType/{title_type_id}

4.2 Key Mapping Decisions

Participation as a reification bridge: The title_principal table is mapped to a Participation class rather than a direct Talent-Title link. This allows a single talent to hold multiple roles and categories within the same title, which a simple binary relationship could not express.

talent_role junction via SQL join: Since talent_role has no title_id column, it cannot be directly joined to Participation. The mapping uses an rr:sqlQuery joining title_principal and talent_role on talent_id, allowing the Participation URI to be constructed and ont:plays to be asserted correctly.

Nullable foreign keys handled via rr:joinCondition: The region_id and language_id columns in title_aka are nullable. Using rr:joinCondition instead of direct templates means rows with NULL values are safely skipped rather than producing broken IRIs.

ContentType and Region promoted to full TriplesMaps: An earlier version of the mapping used direct templates for these, which minted URIs without ever asserting rdf:type. This caused them to appear as homeless individuals in Protege. The fix adds dedicated TriplesMaps for both.

5. SHACL Validation

SHACL (Shapes Constraint Language) is used to validate that the generated RDF data conforms to the structural requirements defined by the ontology. The shapes file (shapes.ttl) defines one NodeShape per class, specifying mandatory properties, cardinality constraints, and expected target classes for object properties.

5.1 Shapes Summary

Shape	Mandatory Properties	Optional Properties
TitleShape	primaryTitle (1), isClassifiedAs (1)	contains, hasGenre, isAka, startYear
TalentShape	talentName (1)	participatesIn
ParticipationShape	as — Category (min 1)	plays, job
AlternativeTitleShape	alternativeTitleName (1)	isFrom, hasType
CategoryShape	categoryName (1)	—
RoleShape	roleName (1)	—
GenreShape	genreName (1)	—
ContentTypeShape	content_type_name (1)	—
RegionShape	regionName (min 1)	—
TitleTypeShape	title_type_name (1)	—

5.2 Validation Result

The validation was executed using Apache Jena SHACL with the following command:

```
shacl validate --shapes "shapes.ttl" --data "out.ttl"
```

The result was:

```
[ rdf:type sh:ValidationReport ;
  sh:conforms true
] .
```

sh:conforms true means every individual in the knowledge graph satisfies all mandatory constraints. Every Title has a primaryTitle and a ContentType. Every Participation has at least one Category. Every dictionary individual (Role, Genre, ContentType, etc.) has its name property populated. No violations were found.

6. SPARQL Queries

A total of 18 SPARQL queries were written to demonstrate the knowledge graph. They are organised into three levels of complexity.

6.1 Simple Queries (S1–S8)

These queries each target a single class and verify that individuals and their data properties are correctly populated.

ID	Query Name	What it tests
S1	Titles and start year	primaryTitle and startYear on Title
S2	All genres	genreName on Genre
S3	All content types	content_type_name on ContentType
S4	All roles	roleName on Role — one of the originally empty classes
S5	All categories	categoryName on Category
S6	All regions	regionName on Region
S7	Titles with content type	isClassifiedAs object property
S8	Alternative titles	isAka object property

6.2 Medium Queries (M1–M4)

ID	Query Name	What it tests
M1	Full cast and roles	Full chain: Title → Participation → Talent → Role + Category
M2	Titles genres and talent count	Title → Genre and Title → Participation with GROUP_CONCAT and COUNT
M3	Talents by genre	Indirect path: Talent → Participation → Title → Genre
M4	AKA regional coverage	Title → AlternativeTitle → Region with aggregation

6.3 Non-Trivial Queries (NT1–NT5)

ID	Query Name	Why non-trivial
NT1	Title profile summary	Two independent COUNT(DISTINCT) aggregations + GROUP_CONCAT + OPTIONAL in one query
NT2	Versatile talents	Exploits the Participation reification bridge; HAVING filter on aggregated role count
NT3	Multilingual AKA titles	Traverses three junction mappings; OPTIONAL for nullable TitleType

NT4	Roles x categories breakdown	Dual traversal of Participation → Role and Participation → Category simultaneously
NT5	Genre x content type distribution	Joins Genre ← Title → ContentType; GROUP_CONCAT collapses content types per genre

6.4 Individual Counts (Q1 Sanity Check)

The following counts were obtained by running the class count query against the generated out.ttl:

Class	Individual Count
Talent	1,441
Participation	587
AlternativeTitle	359
Region	245
Title	174
Role	40
Genre	28
Category	12
ContentType	10
TitleType	8

7. Demonstrator

An interactive web demonstrator was built to allow live execution of SPARQL queries against the knowledge graph without requiring Protege or a command-line tool.

7.1 Architecture

- **Backend:** Python Flask server (app.py) that loads out.ttl into an rdflib Graph at startup and exposes a /query endpoint accepting POST requests with a SPARQL query body.
- **Frontend:** A single HTML page (index.html) with a query selector dropdown, an editable SPARQL textarea, a Run button, and a results table.
- **SPARQL engine:** rdflib — a pure Python RDF library that supports full SPARQL 1.1 including GROUP BY, HAVING, OPTIONAL, and GROUP_CONCAT.

7.2 How to Run

Place app.py, index.html, and out.ttl in the same folder, then run:

```
pip install flask rdflib
python app.py
```

Then open **http://localhost:5000** in any browser. The demonstrator loads all 13 pre-defined queries in the dropdown and allows free-form query editing.

7.3 Files

File	Role
app.py	Flask backend — loads out.ttl, handles SPARQL execution via rdflib
index.html	Frontend — query selector, editor, results table
out.ttl	The generated RDF knowledge graph (output of R2RML mapper)
mapping_fixed_v2.ttl	The R2RML mapping file
shapes.ttl	SHACL shapes for data validation
audiovisual-ontology-M3.owx	The OWL ontology file

8. Results Summary

The following table summarises the outcomes of each project stage:

Stage	Status	Notes
SQL Schema	Complete	13 tables, MariaDB 10.4
OWL Ontology	Complete	10 classes, 9 object properties, 11 data properties
R2RML Mapping	Complete	12 TriplesMaps, all classes and properties covered
RDF Generation	Complete	out.ttl — 10 classes loaded, all non-zero
SHACL Validation	Passed	sh:conforms true — 0 violations
SPARQL Queries	Complete	18 queries: 8 simple, 4 medium, 5 non-trivial + 1 sanity check
Demonstrator	Complete	Flask + rdflib web interface, 13 pre-loaded queries

8.1 Key Challenges and Solutions

Challenge 1 — Homeless individuals (Role, TitleType, ContentType, Region): Four classes appeared empty in Protege because the mapping either used direct IRI templates without type assertions, or used incorrect predicate names mismatched with the OWL. Solution: added dedicated TriplesMaps for ContentType and Region, and corrected all predicate URIs by cross-referencing the OWL file directly.

Challenge 2 — talent_role junction unmapped: The role dictionary was populated but no Talent ever pointed to a Role, because the talent_role junction table was not mapped at all. Solution: added a MapParticipationPlaysRole TriplesMap using an SQL join between title_principal and talent_role to construct the correct Participation URI and assert ont:plays.

Challenge 3 — Nullable foreign keys in title_aka: Direct IRI templates for region_id produced broken URIs when region_id was NULL. Solution: switched to rr:joinCondition which causes the mapper to silently skip rows with NULL join values.

Challenge 4 — RDFLib OPTIONAL aggregation bug: RDFLib raises "Variable not bound" when GROUP_CONCAT is applied to a variable coming from an OPTIONAL block. Solution: wrapped the optional variable in COALESCE(?genreName, "") to ensure it is always bound.

9. Tools and Technologies

Tool / Technology	Version	Role in Project
MariaDB	10.4.32	Source relational database (IMDb data)
phpMyAdmin	5.2.1	SQL export (imdb.sql)
OWL / Protege	OWL 2	Ontology design and individual browsing
R2RML	W3C Rec.	Mapping language SQL → RDF
r2rml.jar	—	R2RML mapper — generates out.ttl from mapping + DB
Apache Jena	6.0.0	ARQ (SPARQL queries) + SHACL validation
SHACL	W3C Rec.	RDF data validation via shapes constraints
Python / rdflib	3.x / 7.x	SPARQL engine in the demonstrator backend
Flask	3.x	Web server for the demonstrator
OpenJDK	25.0.2	Required runtime for Apache Jena

End of documentation.